

CPE 301: Embedded Systems Design

Final Project Part 3 Assignment

Temperature Regulator System

Individual / Group members' names:
Evan Schank

1. Project Motivation/Introduction

The purpose of this project was to design and implement an automatic temperature regulator using an Arduino Mega 2560. Temperature regulation is important in many electronic, mechanical, and environmental systems because excessive heat can reduce efficiency, damage components, or create unsafe operating conditions. This system monitors temperature using a thermistor voltage divider and automatically turns on a motor-driven cooling fan when the measured temperature rises above a selected threshold.

The completed system uses a 75°F trigger temperature. If the temperature is at or below 75°F, the system remains in an idle state with the fan off. If the temperature rises above 75°F, the system enters a cooling state and turns on the motor. An RGB LED is used to clearly indicate the current state of the system. The project also uses push buttons for start, reset, and off control, an interrupt service routine for the start button, register-level ADC reading, and custom UART serial output instead of standard Arduino library functions.

2. Proposed System

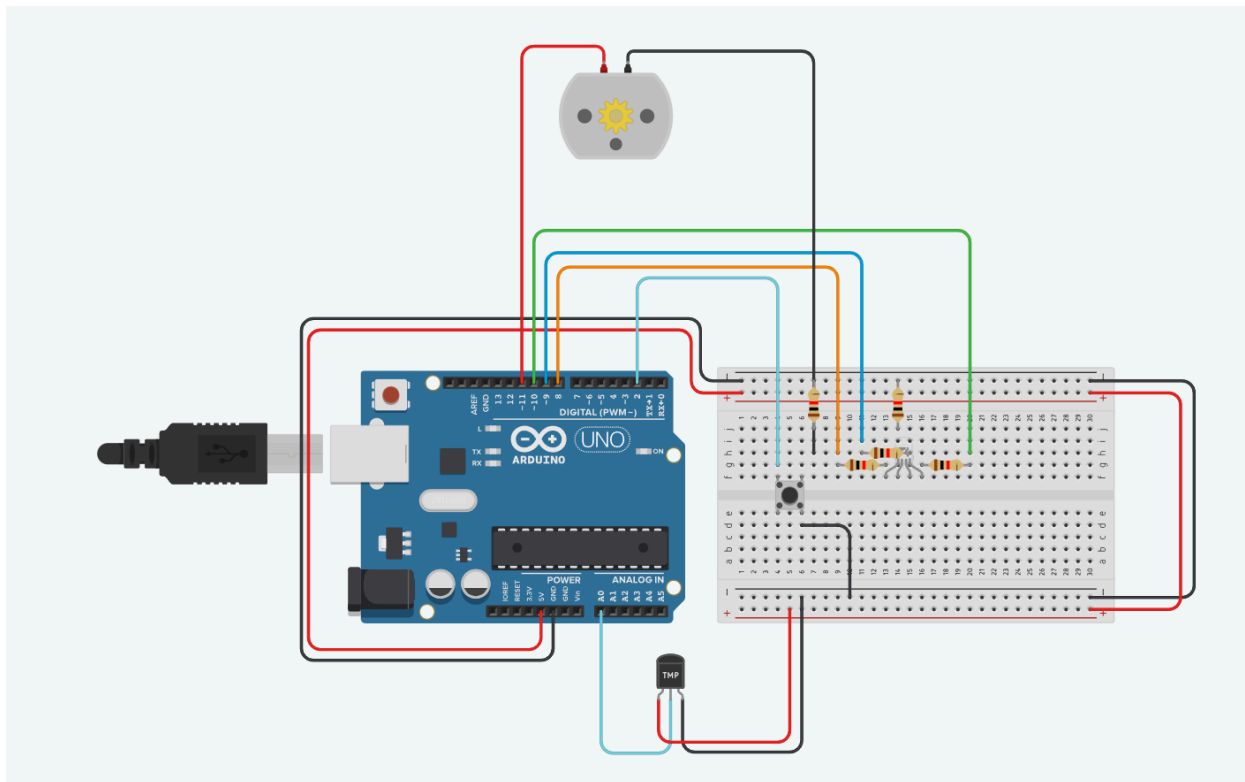
The proposed system is an Arduino-based automatic temperature regulator. The Arduino Mega reads an analog voltage from a thermistor voltage divider connected to A0. This voltage changes based on temperature. The program converts the ADC reading into temperature in degrees Fahrenheit, then compares the measured temperature to the trigger value of 75°F.

The system has four main states: OFF, IDLE, COOLING, and RESET. The RGB LED provides visual feedback for each state. The motor represents the cooling fan.

The system uses the following pin assignments:

A0 = thermistor voltage divider signal
D2 = start button with interrupt
D3 = reset button
D4 = off button
D8 = RGB red
D9 = RGB green
D10 = RGB blue
D11 = Motor

2.1 System Schematic Diagram



The full system contains the Arduino Mega 2560, a thermistor voltage divider, three push buttons, an RGB LED, and a 3.5v motor used to represent the cooling fan.

The thermistor circuit is connected as a voltage divider:

5V
|
[10k NTC Thermistor]
|
+---- A0
|
[10k Fixed Resistor]
|
GND

The buttons are wired using the Arduino's internal pull-up resistors:

D2 ---- Start Button ---- GND
D3 ---- Reset Button ---- GND
D4 ---- Off Button ---- GND

The RGB LED is connected as follows:

D8 ---- resistor ---- RGB red pin
D9 ---- resistor ---- RGB green pin
D10 ---- resistor ---- RGB blue pin
RGB common cathode ---- GND

The motor is connected as follows:

D11 -> Motor Power

Motor GND -> Ground

2.2 Detailed Component List With References

Component	Quantity	Purpose
Arduino Mega 2560	1	Main microcontroller for reading inputs and controlling outputs
10k NTC Thermistor	1	Temperature sensor
10k Resistor	1	Fixed resistor for thermistor voltage divider
RGB LED	1	Indicates the current system state

220 ohm or 330 ohm resistors	3	Current-limiting resistors for RGB LED pins
Push Buttons	3	Start, reset, and off user controls
3.5v motor	1	The cooling fan
Breadboard	1	Circuit construction platform
Jumper Wires	Several	Electrical connections between components
USB Cable	1	Arduino programming and serial monitoring

References:

Arduino Mega 2560 documentation

ATmega2560 datasheet

Thermistor datasheet

Arduino `millis()` timing reference

Arduino interrupt reference

3. System Behavior

The system is controlled using a state machine. The four states are OFF, IDLE, COOLING, and RESET.

In the OFF state, the system is inactive. The RGB LED is off, the motor is off, and the system waits for the start button. The start button is connected to D2 and uses an interrupt service routine. When the start button is pressed, the system moves from OFF to IDLE.

In the IDLE state, the system is active but does not need cooling. The RGB LED turns green and the motor remains off. The thermistor is continuously read using the ADC registers. If the measured temperature rises above 75°F, the system changes to COOLING.

In the COOLING state, the RGB LED turns blue and the motor turns on. The system remains in COOLING while the temperature is above 75°F. Once the temperature falls to 75°F or below, the system returns to IDLE.

In the RESET state, the RGB LED turns red and the motor turns off. After briefly showing the reset state, the system returns to OFF. The reset button gives the user a safe way to clear the system and return it to a known condition.

The off button immediately sends the system to OFF from any active state.

3.1 Module Design

The project is divided into several main modules.

The sensor module reads the thermistor voltage divider connected to A0. The analog signal is converted using ADC register-level programming instead of `analogRead()`. The ADC value is converted into resistance and then into temperature using a thermistor equation.

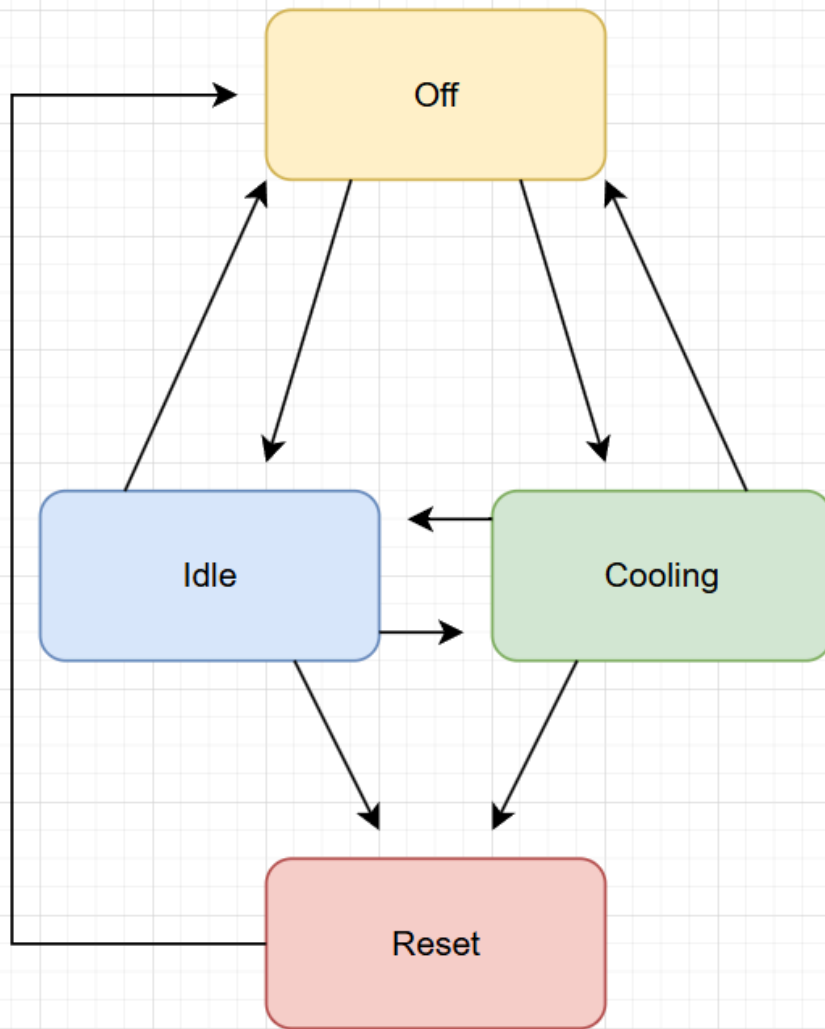
The control module contains the state machine. It determines whether the system should be OFF, IDLE, COOLING, or RESET. The control logic uses the current temperature, the start button, the reset button, and the off button to determine state transitions.

The button module handles user input. The start button is connected to D2 and uses an interrupt service routine. The reset and off buttons are read through direct port access. Internal pull-up resistors are enabled using register manipulation, so the buttons are active-low.

The output module controls the RGB LED and the motor. The RGB LED shows the current state of the system. Green represents IDLE, blue represents COOLING, red represents RESET, and no light represents OFF. The motor is turned on only during COOLING.

The display/logging module sends system data to the Serial Monitor using custom UART functions instead of standard Serial library functions. The output includes the current temperature, system state, and fan state. The timing is handled using `millis()` so the program does not use `delay()`.

3.2 Control Logic / State Diagram



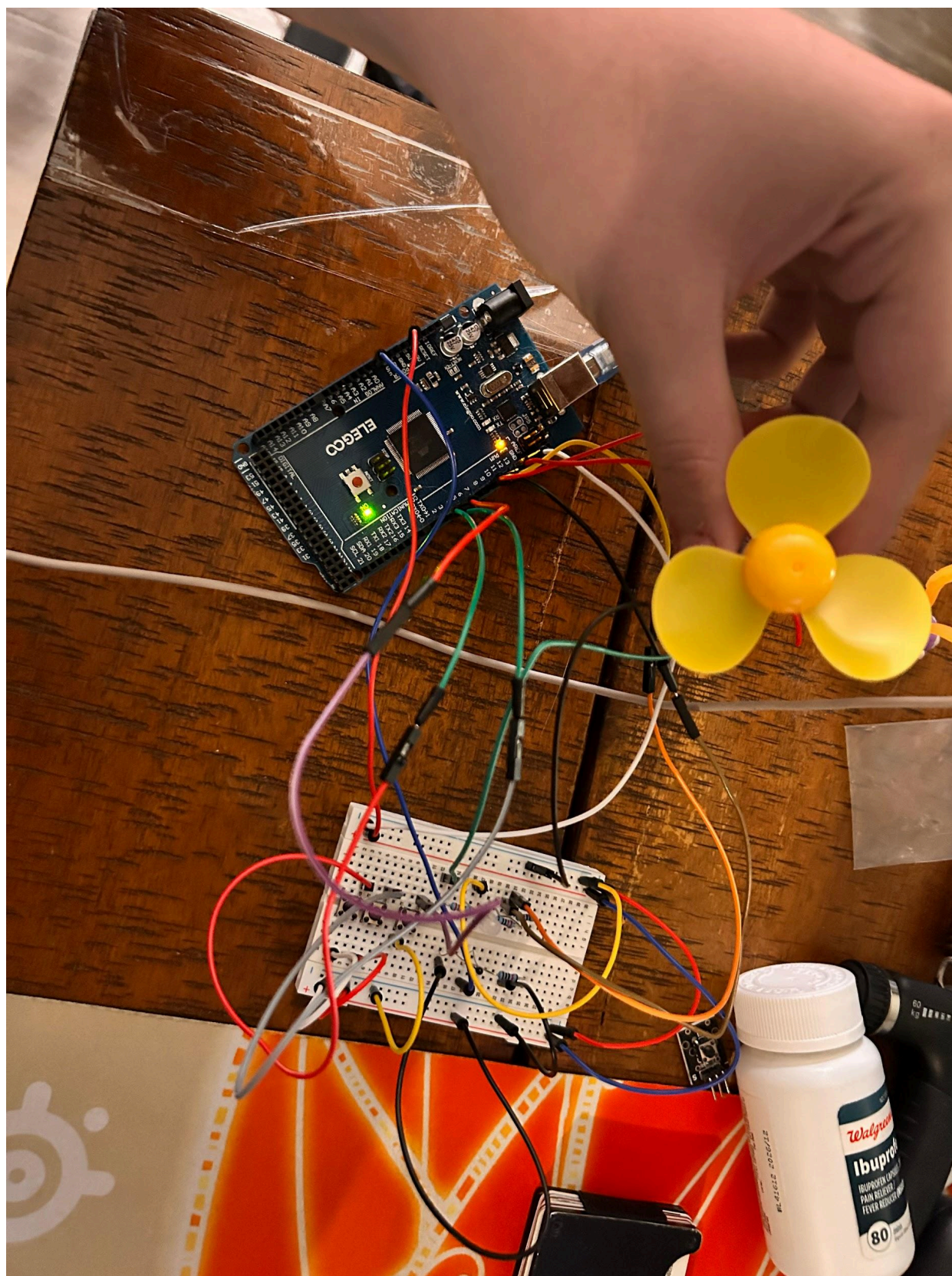
State output table:

State	RGB LED	Motor/Fan	Serial Fan State
OFF	Off	Off	IDLE
IDLE	Green	Off	IDLE
COOLING	Blue	On	COOLING

RESET Red Off IDLE

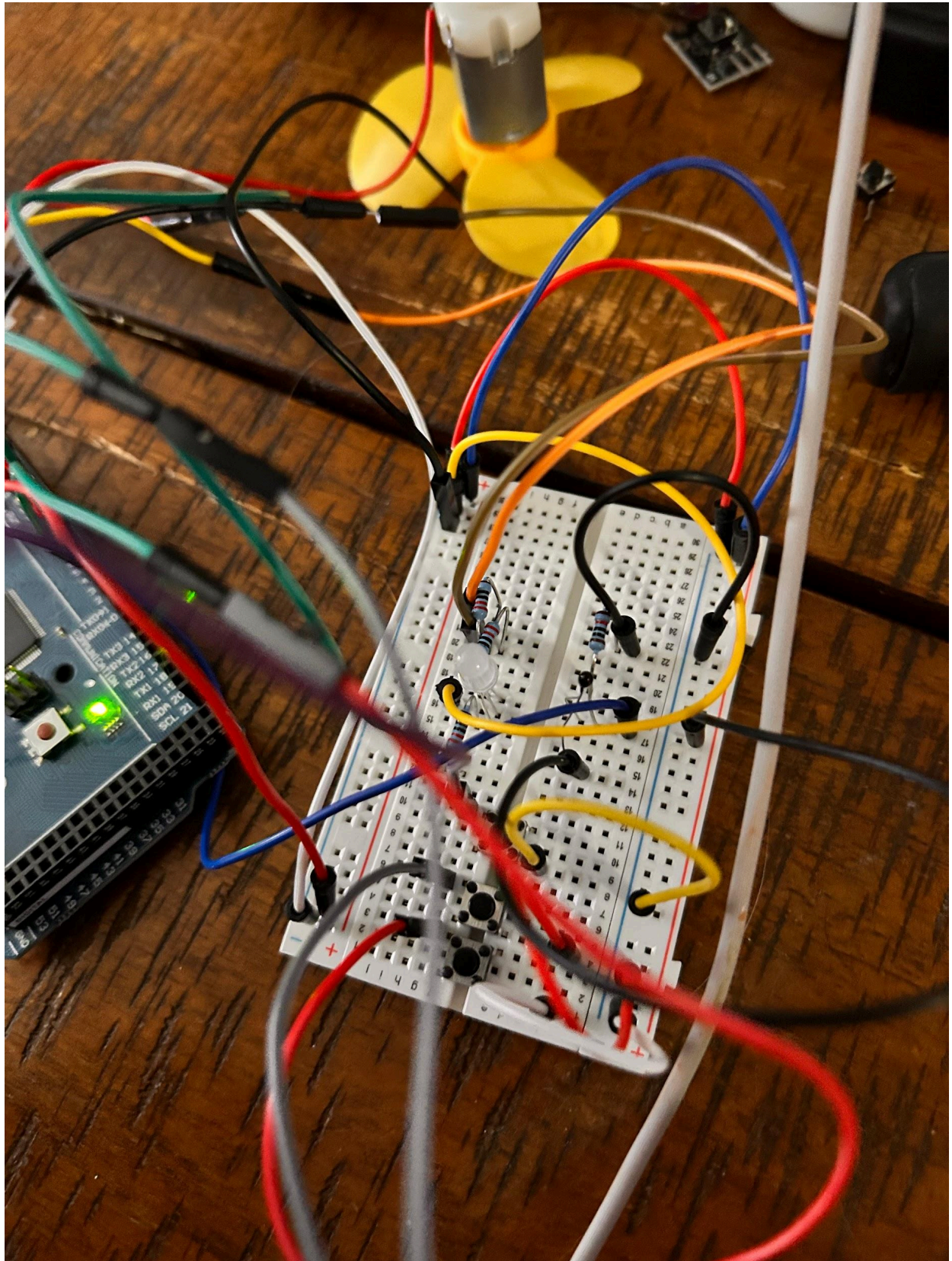
The start button only starts the system from the OFF state. This prevents the start button from interfering with the cooling logic while the system is already active.

4. Result

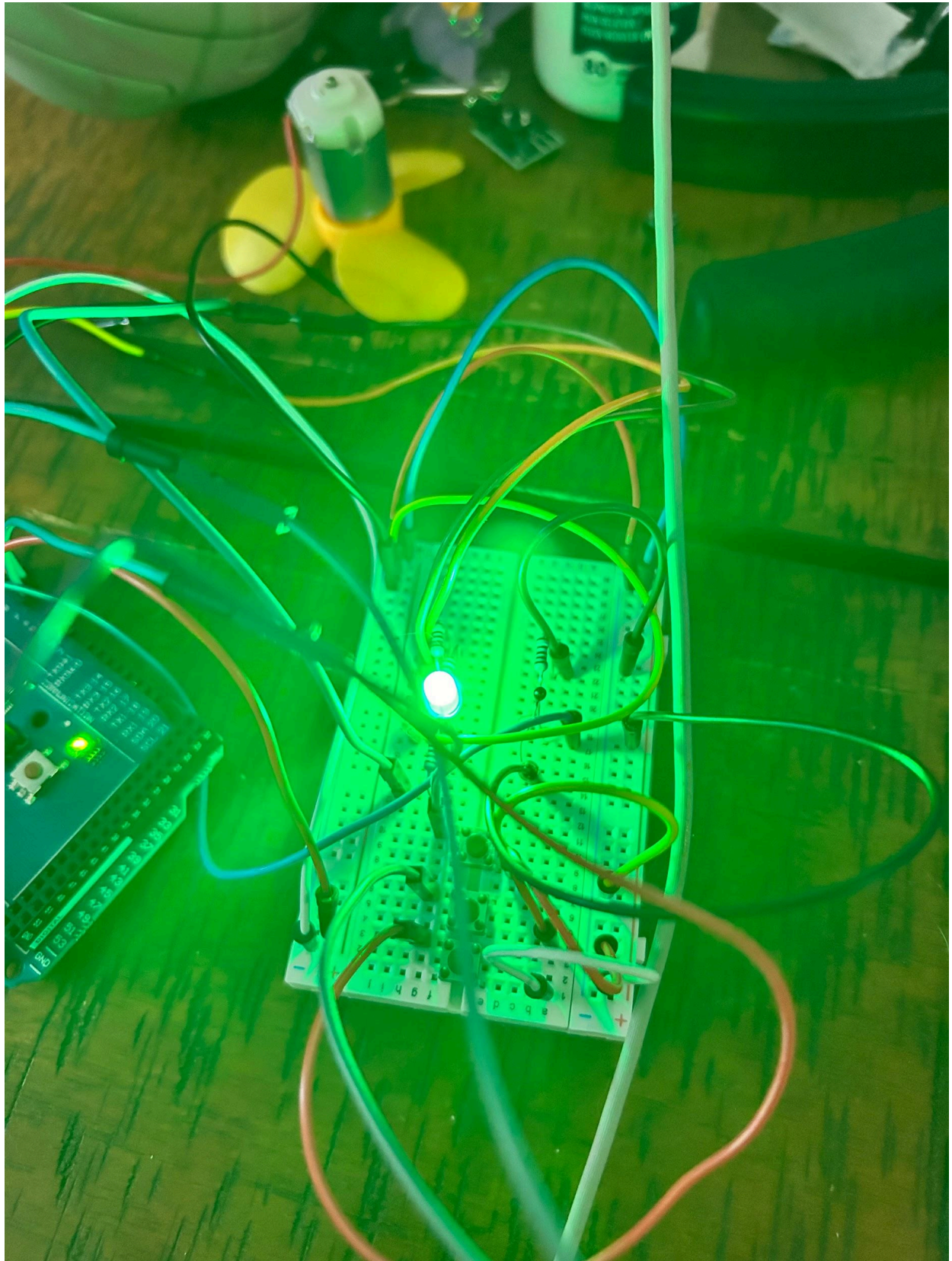


The completed circuit successfully demonstrated the main functions of the temperature regulator. The thermistor voltage divider provided an analog temperature-dependent signal to the Arduino Mega. The Arduino processed this value using register-level ADC code and used the measured temperature to control the system state.

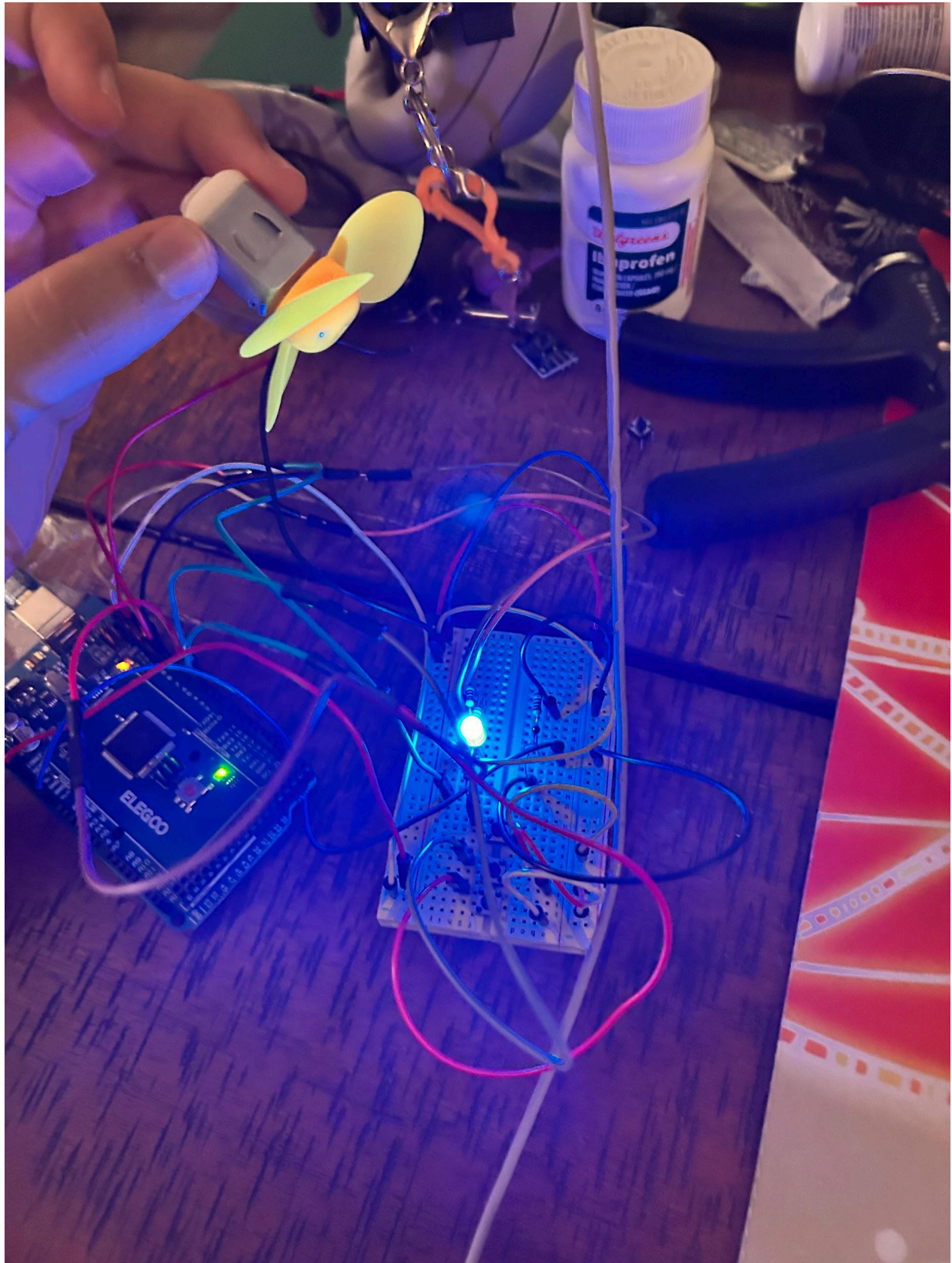
The system used the RGB LED to indicate each state:



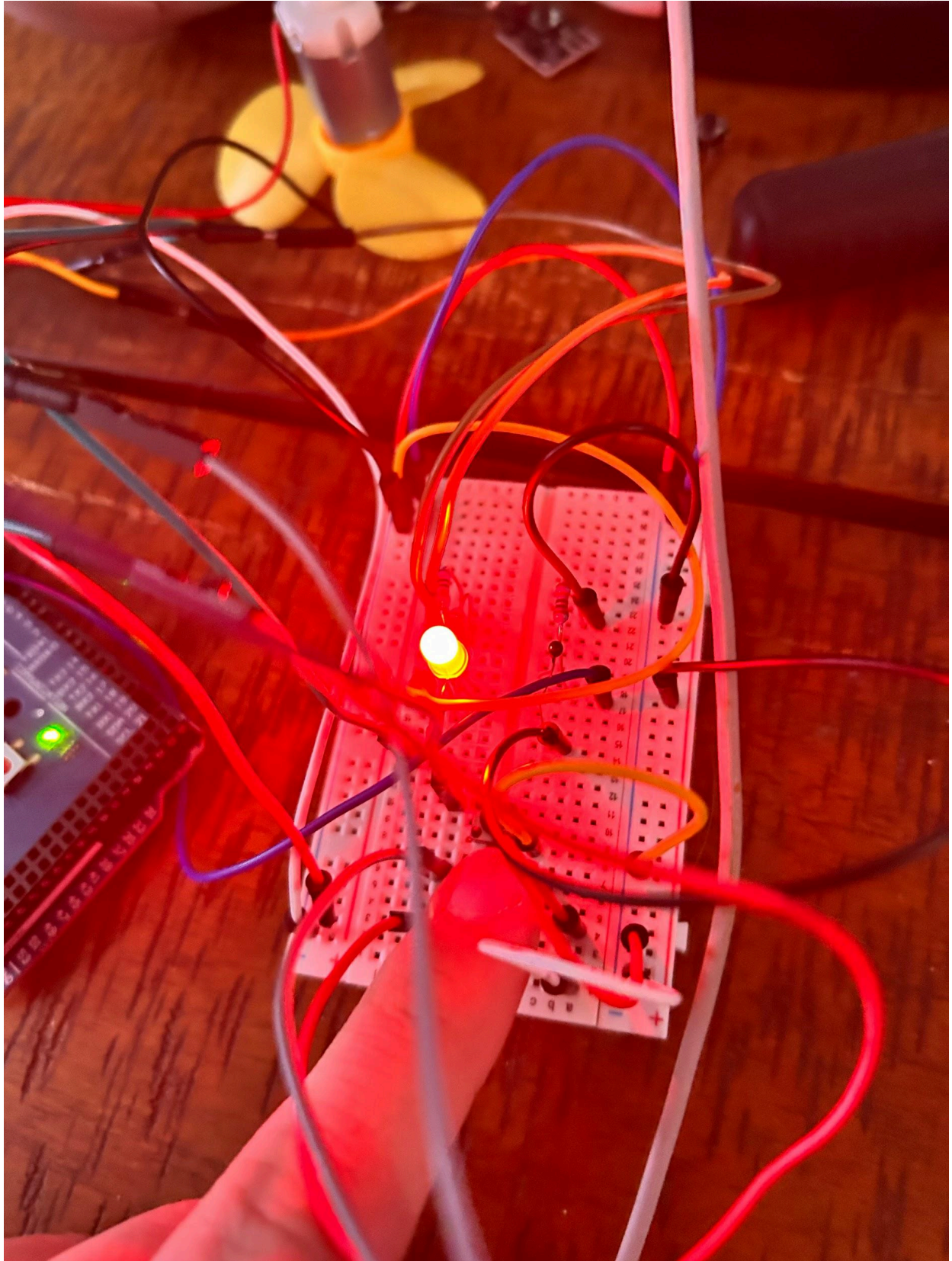
OFF state: RGB LED off, motor off.



IDLE state: green LED on, motor off. This occurs when the temperature is at or below 75°F.



COOLING state: blue LED on, motor on. This occurs when the temperature is above 75°F.



RESET state: red LED on briefly, motor off, then the system returns to OFF.


```
Temperature: 78.1 F | System State: COOLING | Fan State: COOLING
Temperature: 73.7 F | System State: IDLE | Fan State: IDLE
Temperature: 73.1 F | System State: IDLE | Fan State: IDLE
Temperature: 72.8 F | System State: IDLE | Fan State: IDLE
Temperature: 72.6 F | System State: IDLE | Fan State: IDLE
Off button pressed. System OFF.
Off button pressed. System OFF.
Off button pressed. System OFF.
Temperature: 86.8 F | System State: OFF | Fan State: IDLE
Temperature: 91.0 F | System State: OFF | Fan State: IDLE
Start button pressed. System ON.
Temperature: 86.4 F | System State: COOLING | Fan State: COOLING
Temperature: 77.2 F | System State: COOLING | Fan State: COOLING
Temperature: 74.0 F | System State: IDLE | Fan State: IDLE
Reset button pressed.
Reset button pressed.
Start button pressed. System ON.
Temperature: 73.7 F | System State: IDLE | Fan State: IDLE
Off button pressed. System OFF.
Off button pressed. System OFF.
Off button pressed. System OFF.
Temperature: 73.3 F | System State: OFF | Fan State: IDLE
```

The Serial Monitor displayed the measured temperature, the current system state, and the fan state. The output updated once every minute using `millis()` timing. Example output:

```
Temperature: 74.8 F | System State: IDLE | Fan State: IDLE
Temperature: 78.2 F | System State: COOLING | Fan State: COOLING
Temperature: 72.9 F | System State: IDLE | Fan State: IDLE
```

The system avoided the restricted Arduino library functions. The code did not use `pinMode()`, `digitalRead()`, `digitalWrite()`, `delay()`, `analogRead()`, or Serial library functions. Instead, it used direct register access for I/O, ADC, and UART output.

5. Environmental & Energy Impact

The temperature regulator improves energy efficiency by only running the fan when cooling is needed. Instead of powering the motor continuously, the system activates the motor only when the measured temperature rises above 75°F. This reduces unnecessary energy use and helps extend the life of the motor.

The reset button also provides a safe recovery option by turning the motor off and returning the system to a known OFF state.

The project is affordable because it uses common, low-cost components such as a thermistor, fixed resistor, RGB LED, push buttons, and a small DC motor. These parts are widely available and easy to replace.

The design is sustainable because the components are reusable and repairable. If one part fails, such as the motor, LED, or thermistor, it can be replaced without rebuilding the entire system. The system also reduces waste by using simple components instead of requiring a complex prebuilt temperature-control module.

The interface is accessible because it uses clear visual feedback. The RGB LED allows the user to quickly understand the system state: green means safe/idle, blue means cooling, red means reset, and no light means off. The buttons provide simple manual control, and the Serial Monitor gives detailed system information for testing and debugging.

6. Conclusion

The system successfully worked as an automatic temperature regulator. It read temperature data from a thermistor voltage divider, converted the ADC value into a temperature, and used the result to control a motorized cooling fan. The RGB LED clearly indicated the current state, and the Serial Monitor displayed temperature and fan-state information. The start button used an interrupt, while the reset and off buttons allowed the user to control the system safely.

The main limitation of the system is that thermistor readings depend on the accuracy of the thermistor constants and the voltage divider. If the thermistor value, fixed resistor value, or beta coefficient is slightly different from the code assumptions, the measured temperature may not be perfectly accurate. Another limitation is that the motor in the simulation represents a fan, but the actual cooling performance would depend on the size and airflow of the real fan. The system also uses `millis()` for timed logging rather than a dedicated real-time clock module, so it can track elapsed time but not real calendar time.

Future improvements could include adding an LCD display, adding adjustable temperature thresholds, using a real-time clock module for timestamped logging, adding hysteresis to prevent rapid switching near the threshold, and using a stronger fan for real-world cooling applications.